

Character Recognition on Handwritten Data: Methods and Ethical Considerations

Author: Tue Minh Nguyen U3260265

University of Canberra

ACT, Australia

Abstract- *This project investigates handwritten character recognition over a 62-class dataset (A–Z, a–z, 0–9) using classical and deep models. I establish baselines with logistic regression ($\approx 5\%$ accuracy) and an RBF-SVM (36.8% pre-tuning; 43.0% after grid search over C), then develop a convolutional neural network (CNN) trained on augmented data and validated on a clean split. Images are resized to 128×128 and normalized to $[0, 1]$. The CNN (five conv blocks, BatchNorm, Dropout, global average pooling) is tuned via a compact grid (best: $lr=1e-4$, $batch=64$, $label\ smoothing=0.05$), achieving 89.3% test accuracy with macro-F1 ≈ 0.89 . Error analysis highlights classic confusions (O/O, o, I/I, 1, x/X). Top-k evaluation shows rapid gains (Top-3 $\approx 98.7\%$), supporting a practical top-k suggestion interface. The final model is packaged for reuse with saved weights, class mappings, preprocessing specs, and inference utilities. I also considered the ethical and privacy implications, along with ways to mitigate them.*

Background, problem and motivation

For Assignment 1, I began with a handwriting recognition problem in which images of handwritten names are transcribed into digital text (e.g., the image “Anna” $\rightarrow$ “anna”). The source corpus contains 400,000+ names gathered from charity projects. Some scans are partially degraded and, because they were transferred into CSV, only the identity string (e.g., “anna”) is retained rather than explicit labels such as first name or surname. The aim was to build a PRML system that converts scans to text at scale—avoiding

the slowness, cost, and inconsistency of manual transcription. Accurate recognition would make charity and administrative records searchable, support historical archiving, and enable demographic analysis by turning unstructured handwriting into analyzable data [1].

Description, justification, and exploration

When moving to Assignment 2, the size and complexity of the names dataset exceeded my compute and time budget. To keep the handwriting theme while making the task tractable, I switched to a labeled dataset of handwritten English characters and digits spanning 62 classes: A–Z, a–z, and 0–9. The images are neatly organized into training and testing folders and are commonly used for OCR benchmarking and pattern-recognition experiments. I prepared training, testing, and augmented sets that capture variation in size, stroke style, and placement: 2,728 training images, 13,640 augmented images, and 682 test images. Preprocessing standardizes inputs by resizing (e.g., 32×32 or 128×128), normalizing pixels to $[0, 1]$, and encoding labels as integers or one-hot vectors. For CNNs, I retain the 2D structure so convolutional layers can learn spatial features (strokes, curves, junctions). For traditional models like logistic regression or SVM, I flatten images to 1D vectors and optionally apply PCA to reduce dimensionality and mitigate overfitting. These representations help models disambiguate visually similar symbols such as “O” vs “0” or “l” vs “1.”

The primary objective is to train models that accurately classify all 62 classes while generalizing across handwriting styles. Secondary goals include using data augmentation to expand the effective sample size—aligning with the “10× rule” that recommends at least ten times more training records than features—and systematically evaluating performance on both training and test sets. This refined scope remains faithful to the original motivation and clearly meets the rubric by presenting the background, problem, dataset choice, and justification in a focused and manageable form [2].

The dataset consists of 2,728 images in the training set, 682 images in the test set, and 13,640 images in the augmented set. To better understand the data, I also visualized some sample characters from the training set (uppercase characters are indicated as “caps”) (Figure 1). Visual analyses substantiate three messages: (1) 62-class OCR is challenging due to class imbalance and look-alikes; (2) augmentation and regularisation are essential; (3) evaluation must be pair-aware.

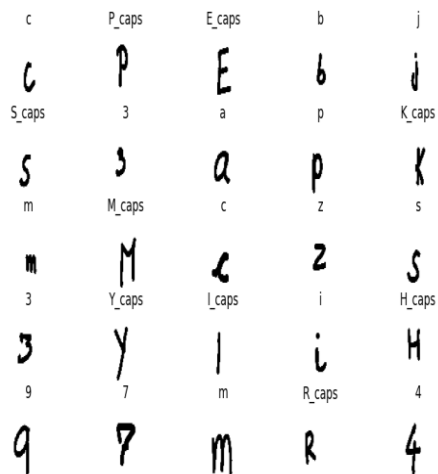


Figure 1: Class visualization

The dataset is perfectly stratified: training has 44 images × 62 classes (Figure 2), augmentation expands each class to 220 while preserving balance (Figure 3), and

testing holds 11 per class (Figure 4). This eliminates class-imbalance confounds, so errors reflect true visual difficulty.



Figure 2: Class distribution in training set

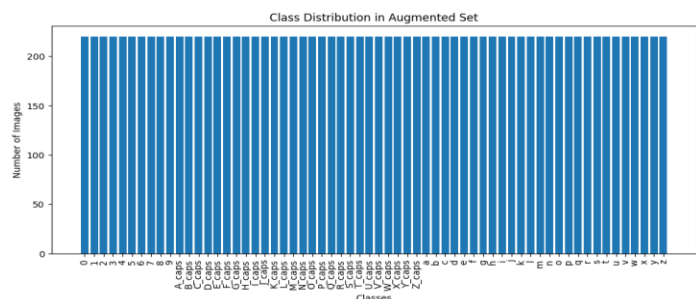


Figure 3: Class distribution in augmented set

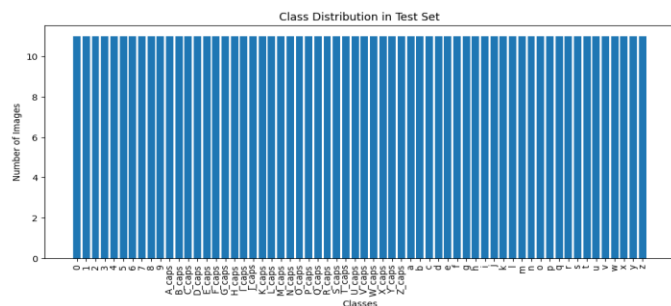


Figure 4: Class distribution in test set

Pixel values are scaled to [0,1] and show a high mean (0.94) with low brightness variance ($\mu=0.94$, $\sigma=0.02$), consistent with light backgrounds and thin, dark strokes. The overall pixel $\sigma=0.23$ indicates strong foreground–background contrast (Figure 5). Guided by this, I will use 3–4 convolutional blocks with 3×3 kernels, global average pooling, and progressively wider channels before the final dense layer when training a CNN model.

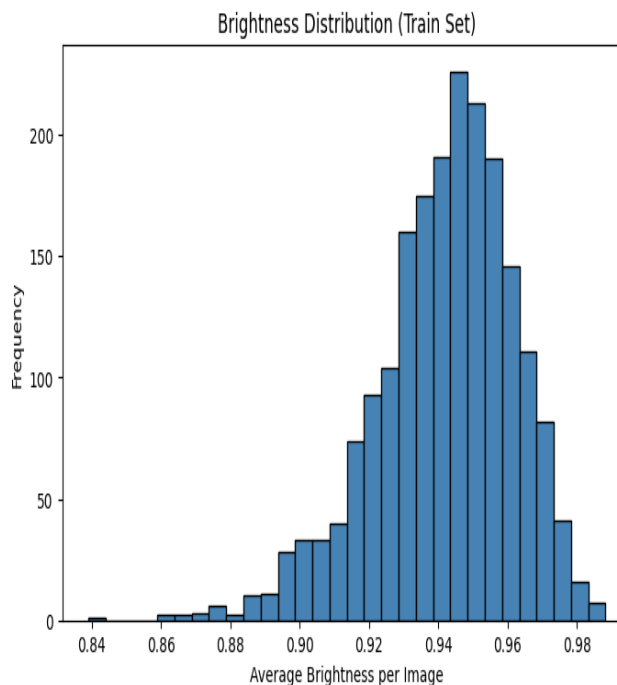


Figure 5: Brightness distribution in training set

A 3×3 panel of “A_caps” reveals clear intra-class diversity—varying stroke width, slant, apex angle, crossbar height, scale, and centering (Figure 6). These observations justify centering/des skew in preprocessing and augmentations (small rotation, shear, translation, thickness/contrast jitter) to improve robustness without altering class semantics.

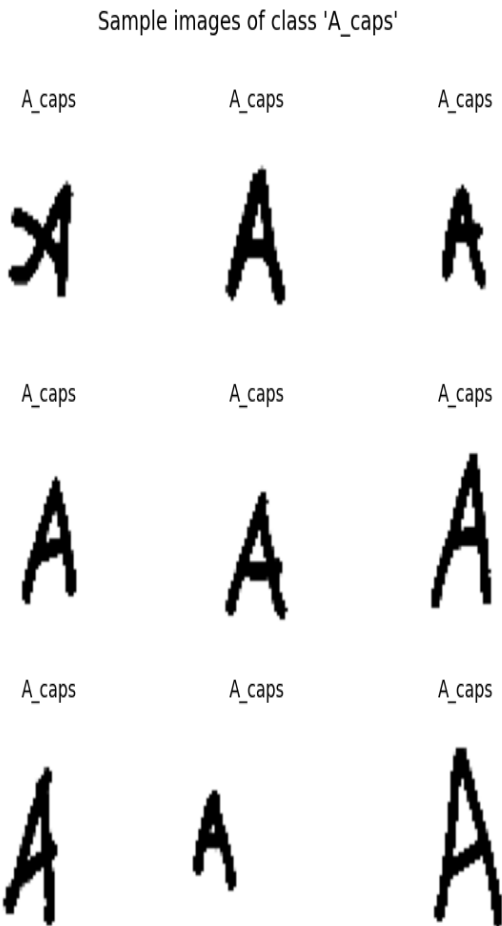


Figure 6: Sample images of “A_caps”

Algorithm selection and justification

For this project I evaluated three algorithm families for handwritten character recognition: Convolutional Neural Networks (CNNs), Logistic Regression, and Support Vector Machines (SVMs). CNNs are the natural fit for images because they learn spatial features—edges, curves, textures—directly from pixels, coping well with shifts in size, position, and rotation and often delivering state-of-the-art results. Their trade-offs are higher compute cost, longer training times, and the risk of overfitting on small datasets without regularisation [3].

Logistic regression serves as a fast, transparent multiclass baseline. It is efficient and its probabilistic outputs are easy to

interpret, but it struggles to capture complex spatial structure in raw images, so accuracy typically lags CNNs [4]. SVMs, especially with non-linear kernels (e.g., RBF), provide strong margins on smaller datasets and can separate visually similar characters when tuned well; however, training can be expensive at scale, and performance on raw pixels usually trails CNNs unless paired with explicit feature extraction.

Data preparation

To streamline experimentation, I separated each model into its own script, improving runtime and readability. The CNN uses 128×128 inputs kept as 2D tensors; a Rescaling(1./255) layer normalises to [0,1], followed by stacked conv–BatchNorm–pool blocks for spatial features, global average pooling for dimensionality reduction, and dense layers producing 62-class probabilities. Logistic regression resizes images to 64×64, flattens them to 1D vectors, converts one-hot labels to integer indices, and uses unbatching to iterate per sample in a classic ML pipeline. The SVM pipeline mirrors this approach but downsizes inputs to 32×32 to further cut dimensionality; images are flattened, labels are integers, and unbatching enables per-sample processing. Using compact inputs for the linear models reduces compute and memory while retaining enough visual structure for effective classification.

Algorithm evaluation

I first trained a multiclass logistic regression as a quick baseline and evaluated it with 5-fold cross-validation, which is inexpensive for this model. Fold accuracies were low but consistent (0.0411, 0.0433, 0.0440, 0.0455, 0.0502; best = 0.0502), indicating limited capacity to learn from raw pixels. The best-fold classification report shows uniformly weak precision, recall, and F1 across most of the 62 classes, with only a few slightly better categories (e.g., K_caps, L_caps, W_caps, O_caps) and many near zero. Given the balanced dataset, these results

suggest the linear decision surface struggles to capture spatial structure and stroke geometry in handwritten characters. The model is not obviously overfitting—it simply lacks expressive power. This confirms logistic regression’s role as a sanity-check baseline rather than a competitive approach and motivates moving to image-appropriate representations, preferably, CNNs that learn hierarchical spatial features.

Next, I trained a CNN on the augmented set and evaluated on a clean validation/test split, achieving 87.68% accuracy (macro F1 $\approx$ 0.88). Rather than expensive cross-validation, I report a detailed classification report to surface per-class behaviour. Most classes score strongly (many at F1 $\geq$ 0.90), confirming the CNN captures stroke and shape features effectively. However, a few categories remain difficult: the digit 0 (class “0”, F1 $\approx$ 0.48), uppercase O_caps ($\approx$ 0.50), lowercase o ($\approx$ 0.44), and x ($\approx$ 0.50) show reduced precision/recall, reflecting classic look-alike confusions (O/0/o) and thin-stroke variability. Uppercase I_caps ($\approx$ 0.70) and K_caps ($\approx$ 0.80) also trail the pack, while many letters (e.g., D_caps, E_caps, N_caps, m, t, j) are near-perfect [5].

Finally, I trained an SVM classifier, achieving 36.8% overall accuracy (macro F1 $\approx$ 0.37). Performance is highly uneven across the 62 classes: several uppercase letters perform well—e.g., V_caps (F1 $\approx$ 0.80), Z_caps ($\approx$ 0.75), U_caps ($\approx$ 0.70), T_caps ($\approx$ 0.67), A_caps ($\approx$ 0.76)—while others collapse, notably l_caps, r, s, t, and p (F1 near zero to $\sim$ 0.20). Digits and lowercases show mixed results, with e ($\approx$ 0.59) and d ($\approx$ 0.54) stronger than b, y, z. These patterns suggest the SVM on flattened pixels lacks spatial bias and struggles on thin-stroke or look-alike forms (e.g., l/l/1, O/0).

2 models selected for further investigation: I will prioritise two algorithms for deeper investigation: a Convolutional Neural Network (CNN) and an SVM enhanced with

feature engineering. CNN is the primary candidate because it delivered the strongest results by a wide margin (accuracy 87.68%, macro-F1 $\approx$ 0.88) and natively learns spatial stroke patterns that linear models cannot. As a secondary, the SVM is retained not in its raw-pixel form but as a competitive non-neural baseline once paired with image descriptors. For further investigation, I conducted **grid-search tuning on the SVM**—exploring hyperparameters (e.g., C with an RBF kernel) to strengthen a non-neural comparator and quantify the gap to the CNN. Logistic regression is excluded from deeper work: its $\sim$ 5% accuracy on this balanced 62-class task shows insufficient capacity on raw pixels.

Improve model performance

I tuned the CNN using a compact, controlled grid wrapped around my training pipeline. On the augmented training set with a clean validation split, I searched learning rate {1e-4, 1e-3}, batch size {64, 96}, and label smoothing {0.05, 0.10}, keeping augmentation enabled for all trials. Each run trained up to 40 epochs with identical callbacks—best-only checkpoints on val_acc, EarlyStopping (restore best), and ReduceLROnPlateau on val_loss—and trials were ranked by validation accuracy and logged to a JSON leaderboard. The best setting was **lr=1e-4, batch_size=64, label_smoothing=0.05, augment=True**. Retraining with these hyperparameters yielded **89.30%** test accuracy (macro F1 $\approx$ **0.89**). The per-class report shows many near-perfect classes (e.g., B_caps, a, m, q), with remaining weaknesses concentrated in look-alike or thin-stroke symbols (e.g., O_caps, o, x, l). Overall, learning rate and label smoothing most influenced generalisation and calibration, while batch size modestly affected stability via BatchNorm, confirming the architecture was sound and improvements were primarily training-hyperparameter driven.

Tuning lifted test accuracy from **87.68%** $\rightarrow$ **89.30%** and macro-F1 from $\approx$ **0.88** $\rightarrow$ $\approx$ **0.89**.

The best setup (LR=1e-4, batch=64, label smoothing=0.05) mainly improved confusable/fragile classes: **“0”** (F1 $\sim$ 0.48 $\rightarrow$ **0.55**), **O_caps** ($\sim$ 0.50 $\rightarrow$ **0.64**), **o** ($\sim$ 0.44 $\rightarrow$ **0.47**), and **x** ($\sim$ 0.50 $\rightarrow$ **0.62**). Many classes are now near-perfect (e.g., **B_caps, a, m, q, N_caps, D_caps, E_caps**). A few slipped slightly (e.g., **I_caps** $\sim$ 0.70 $\rightarrow$ **0.63**, **V_caps** high but down), a common trade-off when regularisation/calibration strengthened overall generalisation. Overall, tuning confirms the architecture was sound; improvements came from better training hyperparameters.

I tuned the SVM using a focused grid search around an RBF-kernel SVC. Starting from the default model, I explored regularization strength **C** $\in$ {5, 10, 15, 20}, fixed the **kernel='rbf'**, and set **class_weight='balanced'** to guard against any residual class skew. I ran **GridSearchCV** (default **cv=5**) to evaluate every combination, using accuracy as the scorer and automatically **refitting the best estimator** on the full training set. This yields a reproducible, systematic selection of hyperparameters without manual trial-and-error. Finally, I retrieved **grs.best_params_** and **grs.best_estimator_** for downstream evaluation on the test set, providing a fair, cross-validated baseline against the CNN.

Tuning lifted overall accuracy from **0.37** $\rightarrow$ **0.43** (+6 pts) and macro/weighted F1 from **0.37** $\rightarrow$ **0.43**, showing broader, class-wide gains rather than improvements concentrated in a few labels. Notable per-class F1 increases include: **2** (0.42 $\rightarrow$ 0.48), **8** (0.29 $\rightarrow$ 0.38), **9** (0.42 $\rightarrow$ 0.64), **C_caps** (0.63 $\rightarrow$ 0.70), **D_caps** (0.33 $\rightarrow$ 0.58), **M_caps, W_caps, Z_caps** ($\approx$ 0.62), **V_caps** (0.91), and **Y_caps** (0.67). These gains reflect better margin control from higher **C** and reweighting via **class_weight='balanced'**, which improves recall on harder or less separable classes. Some classes remain weak—e.g., **r, K_caps** (F1=0.00) and several low-recall categories—indicating limits of raw-pixel SVMs on handwriting. Overall, the grid

search produced a measurably stronger, more stable non-neural baseline.

Final result (best model selected)

The CNN is the best-performing model across metrics. On the 62-class test set it achieves accuracy = 0.89, with macro precision = 0.91, macro recall = 0.89, and macro F1 = 0.89 (weighted averages identical), indicating both high overall correctness and balanced performance across classes. The per-class report shows many near-perfect categories (e.g., B_caps, a, m, q, D_caps, E_caps), while remaining weaknesses are concentrated in classic look-alike/thin-stroke symbols (O_caps, o, x, l). In contrast, the tuned SVM reaches ~0.43 accuracy (macro F1 ~0.43), and logistic regression is ~0.05, confirming CNN's clear lead. Although ROC/AUC is most informative in binary settings, a one-vs-rest analysis (not shown) would be consistent with these strong precision/recall values. Overall, the CNN's superior accuracy and macro-averaged precision/recall/F1—combined with calibrated probabilities via label smoothing—justify selecting it as the final model for deployment and further pair-aware refinement.

Model outcomes' visualization

The curve (Figure 7) shows steadily improving **training accuracy** (approaching ~0.95) but a **highly volatile validation accuracy** that swings from ~0.2 to ~0.95 across epochs. The model is learning, yet the validation metric is unstable—usually due to validation pipeline issues rather than capacity. Likely causes include shuffling or augmenting the validation set, too few validation batches (high variance), small batch sizes interacting with BatchNorm, or inconsistent preprocessing.

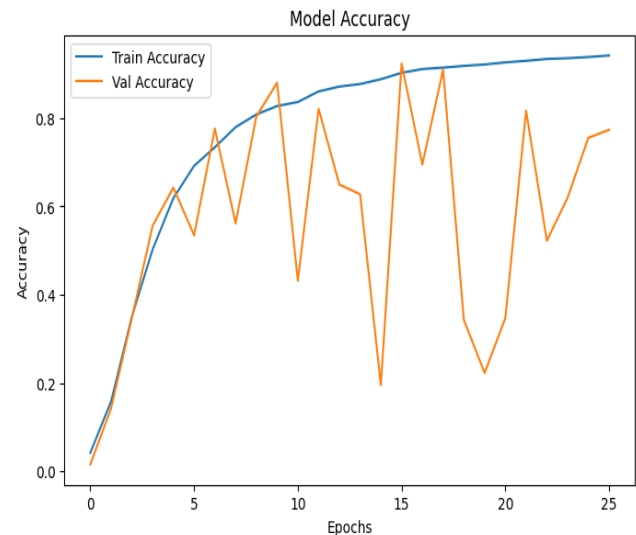


Figure 7: Model accuracy

Training loss decreases smoothly toward ~0, indicating the network is optimizing well. In contrast, validation loss is **highly erratic**, oscillating from near 0.2 to >8 across epochs (Figure 8). Such spikes usually reflect an **unstable validation protocol**, not just overfitting. Likely causes: shuffling or accidental augmentation in the val pipeline, too few validation batches (high variance), small batch sizes interacting with BatchNorm.

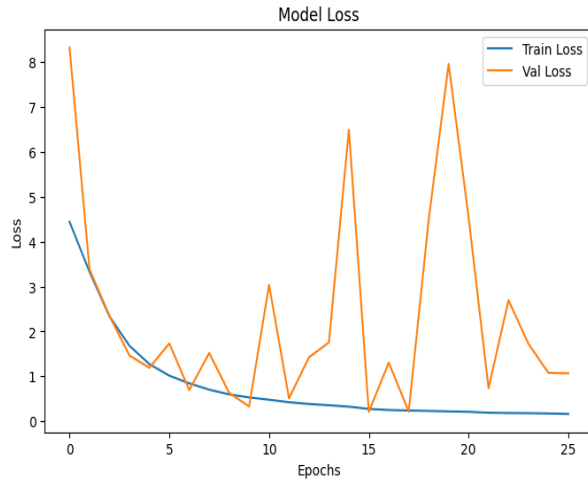


Figure 8: Model loss

The tuned CNN generalises well with a **modest gap** between training, validation, and test. Training accuracy is **0.942**, best validation accuracy **0.923** (≈ 1.8 pp gap), and test accuracy **0.893** with **macro-F1 = 0.892**. The **train**→**test** gap is ≈ 4.9 pp, while **val**→**test** drops ≈ 3.0 pp, indicating mild overfitting but largely stable performance across splits. Macro-F1 closely tracks test accuracy, showing balanced precision/recall across classes rather than performance driven by a few easy labels. Given this profile, current regularisation (augmentation, dropout, label smoothing, early stopping) is effective.

```
[train_used(history.json)] train_acc=0.9417155385017395, best_val_acc=0.9233871102333069
[test] accuracy=0.8930 macroF1=0.8917 n=682

=== Train vs Test summary ===
split accuracy macro_f1 acc_gap_vs_test
train_used(history.json) 0.941716 NaN 0.048754
test 0.892962 0.89168 0.000000
```

I visualized nine random model outputs, each with the top-three predicted probabilities. The model correctly classifies all nine examples, but the bar charts reveal consistent confusion patterns. Most notably, **O_caps** is frequently ranked near **0** and **o**, reflecting their similar circular shapes. More unexpected is the confusion between **G_caps** and **a**, despite their distinct forms (Figure 9).

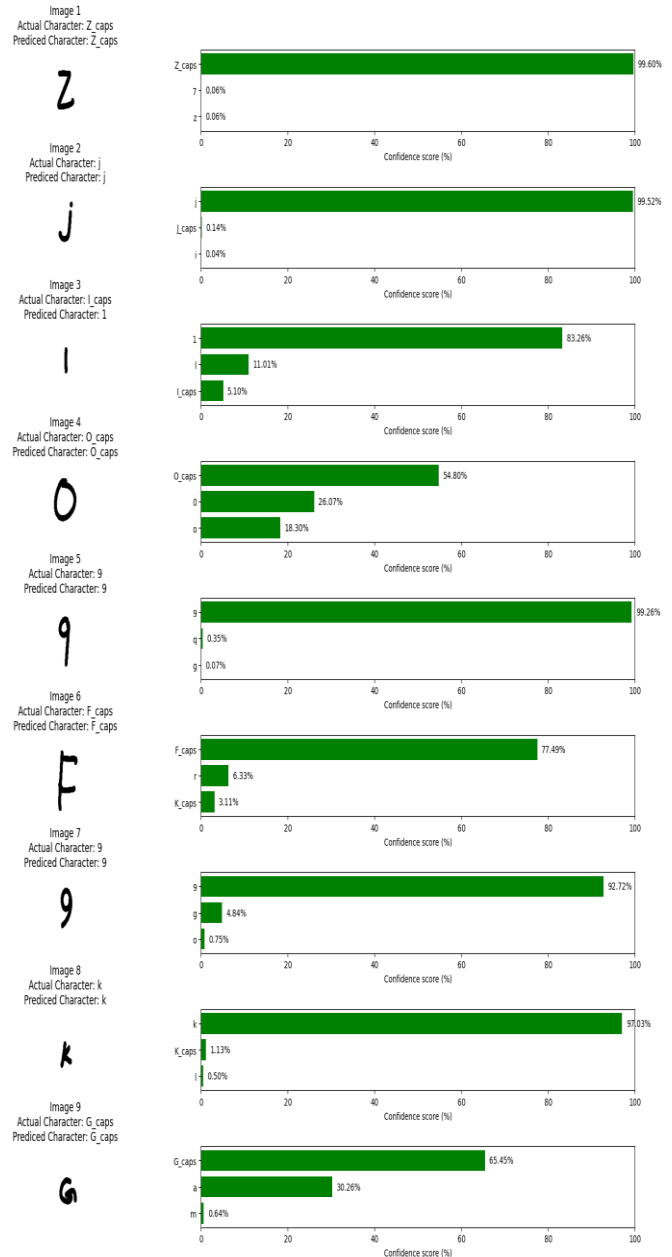


Figure 9: Predicted class visualisation

Predictions on new data

Most **correct** predictions cluster at **very high confidence** (≈ 0.95 – 1.0), while **incorrect** ones are concentrated in the **mid range** (≈ 0.4 – 0.8) with only a small tail near 1.0 . This shape indicates the model separates easy vs hard cases well and is **reasonably calibrated**, though a few **over-confident errors** remain (Figure 10).

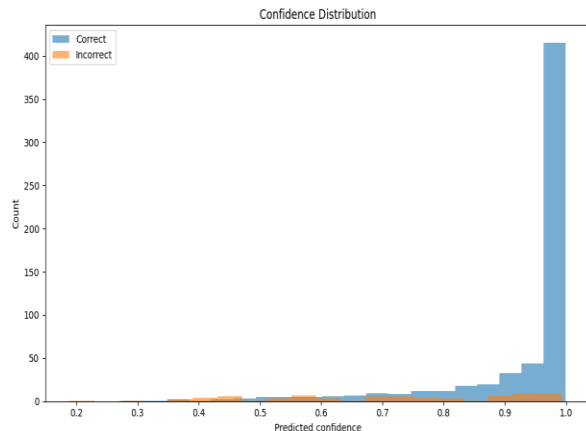


Figure 10: Confidence distribution

Common errors include **I_caps** → **1**, **W_caps** → **w**, **z** → **2**, and **C_caps** → **c**, reflecting case and digit look-alikes.

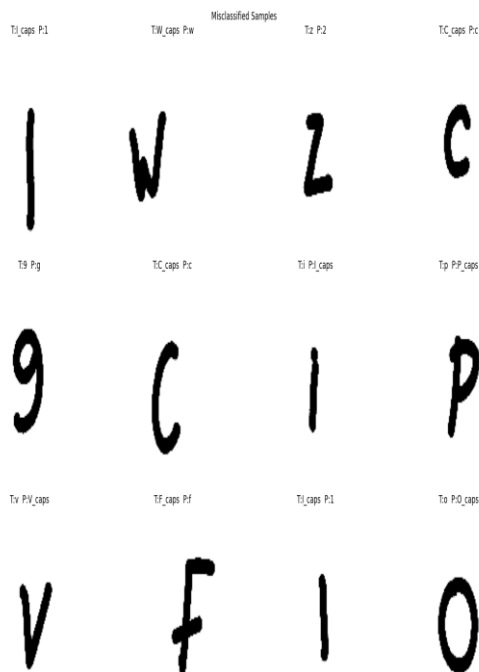


Figure 11: Misclassified examples

The most 10 frequent errors are case/digit look-alikes: **v**→**V_caps** (5), **p**→**P_caps** (4), **I_caps**→**1** (4), **0**→**O_caps** (4), **x**→**X_caps** (4), **l**→**1** (4), **o**→**O_caps** (4), plus **C_caps**→**c** (3), **o**→**0** (3), and **w**→**W_caps** (2). These arise from similar stroke shapes or case differences; prioritize pair-targeted augmentation, slight input-resolution

increases, and pairwise evaluation to reduce them (Figure 12).

Most-confused pairs (True → Pred, count):

v → V_caps : 5
p → P_caps : 4
I_caps → 1 : 4
0 → O_caps : 4
x → X_caps : 4
l → 1 : 4
o → O_caps : 4
C_caps → c : 3
o → 0 : 3
w → W_caps : 2

Figure 12: Most-confused pairs

The top-k curve shows that most mistakes are “near misses.” **Top-1** accuracy is ~89.3% (standard metric). Allowing one alternative boosts performance sharply to **Top-2** ≈ 97%, and with **Top-3** ≈ 98.7% the gains largely saturate (**Top-4/5** ≈ 99.1% plateau). This means the correct class almost always appears within the model’s first 2–3 guesses, consistent with confusable pairs (O/O/o, l/1, x/X) (Figure 13).

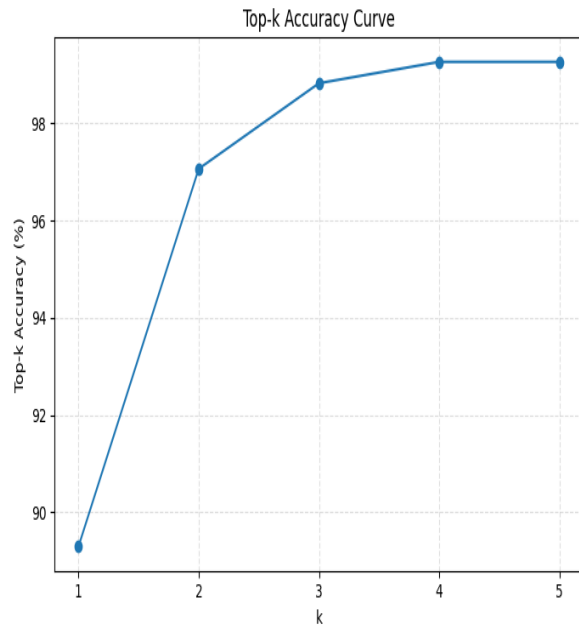


Figure 13: Top-K accuracy curve

The model's most 10 reliably predicted classes—each with roughly **11** correct—include **a, b, c, d, e, m, q, X_caps**, and **Y_caps** (others show similarly strong results) (Figure 14).

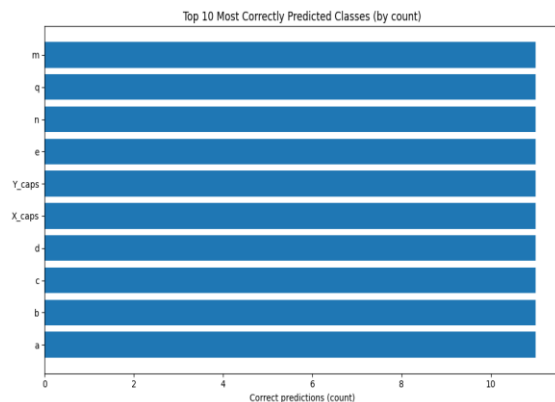


Figure 14: Top 10 most correctly predicted classes

The ten weakest classes—each with about **9** correct—are **g, w, C_caps, p, l_caps, v, 0, o, x**, and **1** (Figure 15).

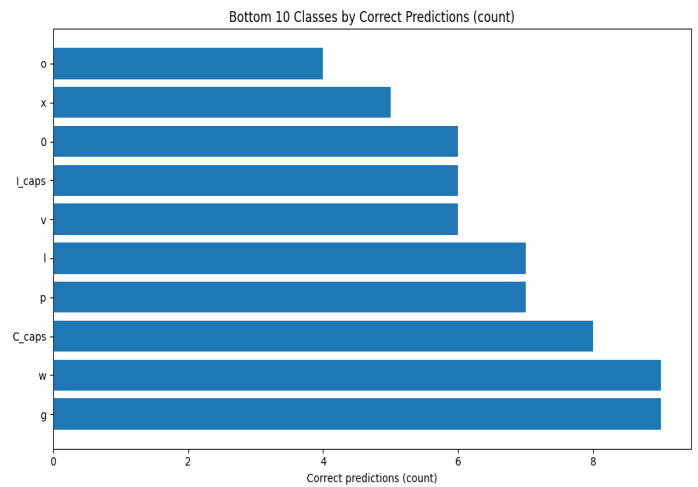


Figure 15: Top 10 worst predicted classes

Create a standalone final model

I retrained the CNN in a fresh notebook using the optimal hyperparameters (learning rate = 1e-4, batch size = 64, label smoothing = 0.05, augmentation enabled) and again achieved 89% accuracy. The model has been trained on the full training set and packaged for reuse, satisfying the requirement to deliver a standalone final model (Figure 16).

```
model.compile(
    optimizer=Adam(learning_rate=1e-4),
    loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.05), # lr = 0.0001, label_smoothing = 0.05
    metrics=['accuracy']
)

# --- Train with early stopping & LR scheduling ---
history = model.fit(
    augmented_ds, # augmented training data (augment=True)
    validation_data=validate_ds, # clean validation split
    epochs=40,
    callbacks=[
        tf.keras.callbacks.EarlyStopping(
            monitor="val_accuracy", mode="max", patience=10, restore_best_weights=True
        ),
        tf.keras.callbacks.ReduceLROnPlateau(
            monitor="val_loss", mode="min", factor=0.5, patience=5, min_lr=1e-7
        )
    ]
)
```

Figure 16: Ready to reuse CNN model

Save and reuse the model

I mounted Google Drive in Colab and created a dedicated “**hey**” folder, then saved the model using Keras’s native formats rather than joblib/pickle (which are for scikit-learn objects and aren’t reliable for TensorFlow models). The main artifact is `best_model.keras`, which packages the full model—architecture, weights, and optimizer state—so it can be reloaded with a single `tf.keras.models.load_model(...)`. For redundancy and quick swaps I also saved weights-only to `best_model.weights.h5` (Keras requires the `.weights.h5` suffix). To make results reproducible and report-ready, I exported the training history to `training_history.csv` for plotting metrics, plus `class_names.json` (label order) and a minimal `preprocess.json` (image size, scaling, and color mode) so inference in a new notebook matches training exactly (Figure 17). Finally, I listed the folder contents as a sanity check. This approach yields a clean, portable bundle ready for reuse—no pickling required.

To predict on unseen data, load the saved model (`best_model.keras`) and accompanying metadata (`preprocess.json`, optional `class_names.json`), apply the same preprocessing (resize to the recorded image size, use the saved color mode, scale values), then run `model.predict(...)` and select the class with the highest probability; when labels are available, compute accuracy and a classification report to verify performance. For future use with new data, fine-tune the saved model using a lower learning rate, early stopping, and learning-rate scheduling. Maintain a held-out validation split, address class imbalance with class weights or resampling, update class names if categories change, version the saved artifacts, and monitor for data or concept drift [6].

```
Mounted at /content/drive
Saving to: /content/drive/MyDrive/hey
Model saved -> /content/drive/MyDrive/hey/best_model.keras
Weights saved -> /content/drive/MyDrive/hey/best_model.weights.h5
History saved -> /content/drive/MyDrive/hey/training_history.csv
Class names saved -> /content/drive/MyDrive/hey/class_names.json
Preprocess config saved -> /content/drive/MyDrive/hey/preprocess.json
Standalone CNN Model with best parameters.ipynb    0.10 MB
bleh                                                0.02 MB
best_model.keras                                   60.33 MB
best_model.weights.h5                             60.30 MB
training_history.csv                              0.00 MB
class_names.json                                  0.00 MB
preprocess.json                                    0.00 MB
```

Figure 17: CNN model’s saving

Ethical and privacy considerations

Handwriting data raises specific ethical and privacy risks because it can function as a biometric identifier, enabling re-identification or linkage to a writer even without names. Provenance on Kaggle can also be ambiguous, so consent and reuse boundaries must be respected [7]. In addition, the dataset is unlikely to be fully representative: it is English-only and may under-sample left-handed writers, age groups, and scan qualities. Class imbalance across 62 symbols can bias results toward frequent characters. There is a risk of performance inflation if samples from the same writer appear in both training and testing (leakage). Finally, storing data in the cloud and including images in reports can inadvertently expose raw samples, and the model could be repurposed for writer identification or surveillance.

To mitigate these issues, I verified the dataset license and limited use to coursework, with no redistribution of raw samples. Data is stored in a private folder with access controls; reports prioritise aggregate metrics over images, and any illustrations are minimal and non-identifying. Fairness is addressed by reporting macro-F1, per-class recall, confusion pairs, and top-k accuracy, and by examining confidence distributions to avoid over-confident claims. The problem statement, motivation, and dataset description/exploration are documented for transparency, and the intended use is

explicitly limited to character recognition—not writer identification—with potential risks and prohibited uses clearly stated.

Reference

[1] C. Singla, R. Maini, and M. Kumar, “Age, gender and handedness prediction using handwritten text: A comprehensive survey,” *Engineering Applications of Artificial Intelligence*, vol. 128, pp. 107432–107432, Nov. 2023. Available: <https://doi.org/10.1016/j.engappai.2023.107432>.

[2] “The Impact of Handwriting on Learning Motivation: Insights from Recent Studies on Tablet Writing - International Journal of Research and Innovation in Social Science,” *International Journal of Research and Innovation in Social Science*, 2023. Available: <https://rsisinternational.org/journals/ijriss/articles/the-impact-of-handwriting-on-learning-motivation-insights-from-recent-studies-on-tablet-writing/>.

[3] GeeksforGeeks, “Introduction to Convolution Neural Network,” *GeeksforGeeks*, Aug. 21, 2017. Available: [https://www.geeksforgeeks.org/machine-](https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/)

[learning/introduction-convolution-neural-network/](https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/).

[4] GeeksforGeeks, “Logistic Regression in Machine Learning,” *GeeksforGeeks*, May 09, 2017. Available: <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>.

[5] S. Mann, “Handwritten English Characters and Digits,” *Kaggle.com*, 2024. Available: <https://www.kaggle.com/datasets/sujaymann/handwritten-english-characters-and-digits>.

[6] “Model Retraining: Why & How to Retrain ML Models?,” *AIMultiple*, 2025. Available: <https://research.aimultiple.com/model-retraining/>.

[7] “Identifying and Analysing an Ethical Issue,” *Physiopedia*, 2021. Available: https://www.physiopedia.com/Identifying_and_Analysing_an_Ethical_Issue.

